01 · Agent 基础与架构

技术面试准备材料 · AI 应用 / Agent 开发

> 核心问题：Agent 到底是什么、怎么运转、架构怎么分层、什么时候不该用 Agent。

1. 什么是 Agent？它和 Workflow、Chatbot 有什么区别？

我理解 Agent 是一个能在环境中自主完成目标的系统：它接收目标，自己决定下一步做什么，调用工具改变环境，观察结果，再继续推理，直到完成或需要人介入。

它和 Chatbot 的区别是：Chatbot 主要产出文本，一次问答就结束；Agent 有目标、有工具、有状态，会执行动作并承担副作用。

它和 Workflow 的区别是控制流的来源：Workflow 的步骤是人预先定义好的，稳定、可复现，但面对变化不灵活；Agent 的步骤是运行时决策出来的，适应性强，但不可控因素更多。工程上我不会把两者对立，通常是 Workflow 负责确定性主干，Agent 负责局部需要判断的环节。

所以在 AI4Test 里，我的做法是：测试流程的主干用固定 Workflow，只有「选择哪个 Agent」「失败后先查什么」这类需要判断的节点才交给 Agent 决策。这样既有灵活性，又能保证可复现和可审计。

2. ReAct 的原理是什么？它有哪些问题？

ReAct 的核心是让模型交替进行 Reasoning 和 Acting：先输出一段思考，决定调用哪个工具、传什么参数，然后观察工具返回，再基于新信息继续思考。形式上就是 Thought → Action → Observation 的循环。

它的优点是实现简单、可解释性比较好，每一步决策都能看到；缺点是错误会沿着链路累积，一步查错后面全偏，而且容易陷入重复调用同一个工具的死循环，成本和延迟也不可控。

工程上我会加几道约束：设置最大步数和 Token/时间预算；对相同工具和参数的重复调用做检测；每步结果做结构化校验；关键节点加验证器或人工确认；一旦置信度低就澄清或转人工，而不是硬跑。ReAct 适合探索性任务，不适合高风险、强确定性的流程。

3. 什么时候该用多 Agent？什么时候不该用？

多 Agent 的价值主要有三个：上下文隔离、并行执行、能力专业化。比如测试平台里，需求分析、用例生成、日志诊断各自需要不同的提示词、工具和知识，放在一个 Agent 里上下文会互相污染，拆开更清晰；能并行的子任务也可以同时跑。

但它不是默认选择。多 Agent 会带来协调成本、错误传播、调试困难和成本上升。如果一个 Agent 加上合适的工具就能完成，或者任务本身是强顺序的，我不会拆。更常见的坏味道是：为了「看起来像架构」而拆出一堆角色，结果每个 Agent 都很薄，通信成本比干活还高。

我的判断标准是：只有当单个 Agent 的上下文确实装不下、任务可以真正并行、或者需要不同的权限边界时，才拆多 Agent。拆的时候也要有统一的 Agent Pool、能力描述和编排层，而不是让 Agent 之间随意对话。

4. Agent 运行时需要管理哪些状态？为什么「跑到一半」很难？

运行时至少要管理五类状态：会话状态（这一轮和上一轮的关系）、执行状态（当前第几步、计划是什么）、上下文状态（这次给模型看哪些消息）、工具状态（哪些调用已发出、结果是否回填）、预算状态（步数、时间、Token、成本）。

「跑到一半」难，是因为它不是一个干净的状态。最典型的是：模型决定调用一个有副作用的工具，工具已经执行了，但结果还没写回，进程就挂了。如果按检查点重跑，可能重复写文件、重复下单；如果不重跑，又可能漏掉结果。

我的处理是把「执行中」和「结果不确定」显式建模：工具调用超过租约还没有回执，就标记为 `uncertain`；恢复时先核对工具状态，再决定重放、跳过还是转人工，而不是只看检查点里有没有快照。同时对关键操作做幂等，让重试本身是安全的。这部分是分布式系统里的老问题，只是换到了 Agent 场景。

5. 一个生产级 Agent 系统应该怎么分层？

我一般分成五层。

最底层是模型接入层，收敛不同厂商的 API 差异，只负责「这个请求怎么写」；上面是模型网关，负责「用哪个模型、挂了怎么切、限流和成本怎么控制」。

中间是 Agent 运行时，管理一次任务的状态、上下文、工具调用、预算和中断恢复。再往上是编排层，根据任务选择 Agent、组织执行顺序，处理并行、依赖和人工介入。

旁边还有两个横切能力：工具层负责注册、权限、沙箱和执行可靠性；可观测与评测层负责轨迹、指标、回归和归因。

这样分的关键是依赖方向单向：底层不依赖上层业务，编排层不关心模型厂商，工具层不关心是哪个 Agent 在调用。分层会增加链路长度和排查成本，所以每一层都要有结构化日志和执行轨迹，出了问题能沿着链路往回追。

6. FSM 和 LangGraph 是什么关系？为什么需要状态机？

FSM 描述的是业务状态和合法迁移，比如一个任务从 PENDING 到 RUNNING、WAITING_APPROVAL、SUCCEEDED、FAILED，哪些迁移允许、哪些必须带条件。它解决的是「当前处于什么状态、下一步能去哪」。

LangGraph 更像是执行图和运行时，负责节点怎么连、状态怎么在节点之间传递、在哪里做检查点。

两者不冲突，通常是配合使用：外层用 FSM 管业务生命周期，内层用图或循环管一次执行。状态机的价值在于把隐式状态显式化，避免出现「任务看起来在跑，其实已经卡死」这种问题；同时也让恢复、取消、重试有明确的落点。

如果没有状态机，Agent 的所有中间状态都藏在代码和消息历史里，出了问题是不可解释的。尤其是长任务和需要人工审批的场景，状态机基本是必需的。

7. 什么是上下文工程？上下文超限了怎么办？

上下文工程就是决定「这一次给模型看什么」。它不只是把历史消息拼起来，而是按优先级选择：系统指令和约束最高，当前任务状态其次，最近的相关对话再次，工具结果和检索内容按相关性裁剪，历史经验只在需要时注入。

上下文超限时，我一般按几个层次处理：先做确定性裁剪，比如截断过长的工具输出、只保留结构化摘要；再做会话级摘

要，把早期对话压缩成任务状态和关键结论；然后做检索式记忆，把长期信息放到外部存储，需要时再取；最后才考虑换更长上下文的模型。

这里最关键的是压缩不可逆：摘要一旦丢了细节，后面就找不回来。所以我会把原始消息、压缩结果和压缩策略都记录下来，关键任务状态单独保存，不依赖模型从摘要里回忆。压缩的触发点也要明确，不能等到报错才处理。

8. 如何控制 Agent 的自主性和风险?

我会把自主性分级，而不是简单的「全自动 / 全人工」。

低风险动作，比如只读查询、检索、生成草稿，可以自动执行；中风险动作，比如写文件、改配置，需要参数校验和沙箱；高风险动作，比如删除数据、发布上线、资金操作，必须审批，而且是动作级审批，不是权限级审批。

技术上靠四件事兜底：工具白名单和最小权限；参数 Schema 校验和注入防护；沙箱限制文件、网络和资源；预算限制步数、时间和成本。再加一条：所有决策和执行都要有审计日志，能回答「谁在什么上下文下做了什么」。

自主性不是越高越好。工业场景里，可解释、可回滚、可追责比「看起来智能」重要。我的原则是：让 Agent 在明确边界内尽量自主，边界之外必须有人。

9. Agent 的任务规划能力怎么提升？分解失败怎么办?

提升规划能力，首先是把计划结构化，而不是让模型自由发挥。我会定义计划 Schema：步骤、依赖、输入输出、所需工具、验收标准。然后用少样本示例告诉模型什么样的计划是好的，再用确定性校验检查依赖是否闭合、工具是否存在、参数是否齐全。

分解失败时，不要直接重试同一套提示词。我会先判断失败类型：是目标理解错了、缺少信息、工具能力不匹配，还是计划本身不合法。目标不清就澄清；缺信息就先检索或提问；工具不匹配就换 Agent 或换路径；计划不合法就让模型基于校验错误修复，而不是整份重来。

另外要有兜底：复杂任务可以退回到预定义 Workflow，或者只自动完成前几步、后面转人工。规划能力的上限往往不是模型，而是任务本身的可分解性和系统能提供的工具边界。

10. 怎么判断一个 Agent 系统能不能上线?

我会从五个维度判断。

第一是效果：在带业务标注的评测集上，任务成功率、关键步骤覆盖率、人工修改率是否达到约定标准，而不是只看几个 demo。

第二是可靠性：失败是否可恢复，工具是否有幂等，长任务能否断点续跑，上游故障是否有降级和熔断。

第三是安全与合规：权限是否最小化，高风险动作是否有审批，输入输出是否有 Guardrails，审计是否完整。

第四是成本与性能：单任务时延、Token 成本、并发能力是否在预算内，峰值是否会拖垮系统。

第五是可运维性：有没有执行轨迹、指标、告警、回滚和灰度能力，出问题能不能在可接受时间内定位。

这五条里任何一条明显不达标，我都会认为还不具备上线条件。上线不是「功能跑通」，而是「出了问题能兜住」。

11. 讲一个你设计的 Agent 系统架构（项目题）

我以 `AI4Test` 为例。它的目标是把测试全流程里可标准化的环节交给 `Agent`，包括需求理解、测试点设计、用例生成、环境准备、执行和结果分析。

整体分四层：模型接入与网关层负责多厂商适配、路由、限流和成本；`Agent` 运行时负责单次任务的状态、上下文、工具调用和中断恢复；编排层维护 `Agent Pool`，根据任务能力标签选择 `Agent`，并生成或复用 `Workflow`；最上面是业务层，把测试流程拆成可验收的节点。

可靠性上，工具调用有幂等键和执行记录，长任务有检查点和租约，恢复时先核对「上次那个调用到底做完没有」。效果上，我们维护带业务标注的评测集，记录每次执行轨迹，失败时按「需求理解 → 检索 → 规划 → 工具 → 环境 → 校验」逐层归因。

这个架构最关键的设计是「模型只负责提议，系统负责执行和校验」。模型可以决定下一步做什么，但能不能做、怎么做、做完算不算成功，由确定性代码和业务规则决定。这样既保留了 `Agent` 的灵活性，又不会把生产系统的可靠性押在模型发挥上。

12. 面试官可能追问：Agent 会不会只是「套壳」？你怎么看？

我认为「套壳」和「系统」的区别不在用不用大模型，而在有没有工程化的约束和闭环。

如果只是把用户输入拼进 `Prompt`、把模型输出直接返回，那确实是套壳，谁都能做，也没有稳定性可言。真正的系统要解决的是模型之外的问题：工具怎么安全执行、失败怎么恢复、上下文怎么管理、效果怎么评测、成本怎么控制、出了问题怎么归因。

这些工作不性感，但决定了 `Agent` 能不能进生产。我的项目里大量时间花在幂等、状态恢复、评测集和可观测性上，而不是调 `Prompt`。模型能力会持续变强，但这些工程能力不会因为换模型就消失，反而是长期壁垒。

所以我的判断是：模型是发动机，但能不能上路取决于底盘、刹车和仪表盘。`Agent` 开发的价值，很大一部分在后者。

高频追问

- ReAct、Plan-and-Execute、Reflection 三种模式分别适合什么任务？
- 多 Agent 之间共享上下文还是隔离上下文？各自代价是什么？
- 如果模型每次都选错工具，你会改 Prompt、改工具描述，还是改路由？
- Agent 的状态应该存在内存、Redis 还是数据库？为什么？
- 如何区分「模型能力不足」和「系统设计有问题」？
- 长任务执行到第 20 步失败，你会重跑还是续跑？依据是什么？